



Label augmented and weighted majority voting for crowdsourcing



Ziqi Chen ^a, Liangxiao Jiang ^{a,b,*}, Chaoqun Li ^{c,*}

^a School of Computer Science, China University of Geosciences, Wuhan 430074, China

^b Key Laboratory of Artificial Intelligence, Ministry of Education, Shanghai 200240, China

^c School of Mathematics and Physics, China University of Geosciences, Wuhan 430074, China

ARTICLE INFO

Article history:

Received 29 January 2022

Received in revised form 14 May 2022

Accepted 17 May 2022

Available online 21 May 2022

Keywords:

Crowdsourcing learning

Label integration

Label augmentation

Label weighting

Majority voting

ABSTRACT

Crowdsourcing provides an efficient way to obtain multiple noisy labels from different crowd workers for each unlabeled instance. Label integration methods are designed to infer the unknown true label of each instance from its multiple noisy label set. We argue that when the label quality is higher than random classification, the more the number of labels, the better the performance of label integration methods. However, in real-world crowdsourcing scenarios, each instance cannot obtain enough labels for saving costs. To solve this problem, this paper proposes a novel label integration method called label augmented and weighted majority voting (LAWMV). At first, LAWMV uses the K -nearest neighbors (KNN) algorithm to find each instance's K -nearest neighbors (including itself) and merges their multiple noisy label sets to obtain its augmented multiple noisy label set. Then, the labels from different neighbors are weighted by the distances and the label similarities between each instance and its neighbors. Finally, the integrated label of each instance is inferred by weighted majority voting (MV). The experimental results on 34 simulated and two real-world crowdsourced datasets show that LAWMV significantly outperforms all the other state-of-the-art label integration methods.

© 2022 Elsevier Inc. All rights reserved.

1. Introduction

Supervised learning [9,37] is a very popular machine learning paradigm and is widely applied in the real world. In supervised learning, large-scale labeled data is essential for improving the learning model's quality. In recent years, due to artificial intelligence has developed rapidly, the demand for large-scale labeled data has increased accordingly. However, large-scale labeled data is hard to obtain. Traditionally, a labeling task is performed by domain experts or well-trained workers. This ensures the correctness of labels, but it is expensive and time-consuming [13,35,6].

Fortunately, crowdsourcing provides an economical and efficient solution [38]. In crowdsourcing scenarios, the labels are obtained from crowd workers through online crowdsourcing platforms [1] such as Crowdflower¹, Clickworker² and Amazon Mechanical Turk (AMT)³. This way can quickly obtain labels for large-scale data at a relatively low cost and has been widely

* Corresponding author.

E-mail addresses: ziqichen@cug.edu.cn (Z. Chen), ljjiang@cug.edu.cn (L. Jiang), chqli@cug.edu.cn (C. Li).

¹ <http://www.crowdflower.com>

² <http://www.clickworker.com>

³ <http://www.mturk.com>

applied in domains such as natural language processing, image classification and object detection. However, due to the personal factors of each crowd worker, such as preference, intention and expertise, the label quality is not guaranteed [23,2]. Training models directly using these noisy labels may cause poor performance. To improve the label quality, researchers have proposed many effective task assignment strategies [31,10,30], and after task assignment, how to infer the true label of each instance from its multiple noisy label set obtained from different crowd workers is also a very meaningful topic [22,29,12].

In recent years, label integration for inferring the true label of each instance from its multiple noisy label set has been extensively studied. Majority voting (MV) is the simplest but often effective label integration method. However, MV naively assumes that each crowd worker has the same labeling quality. Obviously, this is almost impossible in the real world and thus hurts the performance of MV. To improve the performance of label integration, researchers have proposed many relatively complex methods from different aspects. For example, [4] proposed the ZenCrowd (ZC) method, which models each crowd worker's reliability through an expectation–maximization method. [11] proposed the KOS method, which models each crowd worker's reliability through a similar belief propagation method. [14] proposed the Iterative Weighted Majority Voting (IWMV) method, which uses the integrated labels obtained by MV to estimate each crowd worker's labeling quality through an iterative method. [41] proposed the Ground Truth Inference using Clustering (GTIC) method, which uses the features generated from multiple noisy label sets to obtain integrated labels by clustering. Recently, [7] proposed the Multiple Noisy Label Distribution Propagation (MNLDP) method, which solves the problem of label integration by propagating the label distributions of instances. Although these methods generally perform better than MV, for better application in the real world, they still need to be further improved.

In this paper, we argue that when the label quality is higher than random classification, the more the number of labels, the better the performance of label integration methods [24,7]. However, in real-world crowdsourcing scenarios, each instance cannot obtain enough labels for saving costs. Therefore, we consider augmenting the multiple noisy label set of each instance without increasing the cost, and KNN can do exactly that. In some other research fields such as image retrieval [16,19] and image understanding [17,18], good experimental results have also been achieved by exploring neighbors' labels. Therefore, we are more convinced that adapting KNN to crowdsourcing scenarios can solve the problem of insufficient labels and propose a novel label integration method called label augmented and weighted majority voting (LAWMV). In LAWMV, we first use the K -nearest neighbors (KNN) algorithm to find each instance's K -nearest neighbors (including itself) and merge their multiple noisy label sets to obtain its augmented multiple noisy label set. Then, we weight the labels from different neighbors by the distances and the label similarities between each instance and its neighbors. Finally, we infer the integrated label of each instance by weighted MV. To sum up, the main contributions of this paper include:

1. The insufficient label problem for saving cost in the real-world crowdsourcing scenarios is transformed into how to use existing information to augment labels, which provides a new perspective to solve the label integration problem.
2. A novel label integration method is designed, which uses the KNN algorithm to find each instance's K -nearest neighbors (including itself) and merges their multiple noisy label sets to obtain its augmented multiple noisy label set, thereby weakens the influence of the lower quality labels in the multiple noisy label set.
3. The distances and the label similarities between each instance and its neighbors are used to weight the labels from different neighbors to enhance the impact of neighbors that are more likely to belong to the same class as each instance.

The rest of this paper is organized as follows. Section 2 describes the related work on label integration. Section 3 introduces the proposed LAWMV in detail. Section 4 reports the experiments and results on simulated and real-world datasets. Section 5 summarizes this paper and outlines the research directions of future work.

2. Related Work

MV is the simplest but often effective label integration method. In MV, the label with the highest frequency is selected as the unknown true label. However, MV naively assumes that each crowd worker has the same labeling quality, which is almost impossible in the real world and thus weakens its performance. In recent years, label integration from multiple noisy label sets has been extensively studied for improving its performance, and a large number of label integration methods have been proposed from different aspects. We divide these label integration methods into two categories based on whether data features are used.

The first category does not use data features, which only relies on multiple noisy label sets to infer unknown true labels of instances. We first introduce the improved MV methods. For example, IWMV (Iterative Weighted Majority Voting) [14] first uses MV's initial integrated labels and multiple noisy label sets to estimate each crowd worker's labeling quality. Then it updates each crowd worker's weight and the integrated labels of instances and iterates multiple times until convergence or reaches the number of iterations. [25] uses the certainty information of the majority class in multiple noisy label sets and thus proposes four soft MV methods. [28] uses multiple noisy label sets to evaluate each crowd worker's labeling quality by a differential evolution method. In addition to the improved MV methods, there are some other methods that do not use data features. For example, DS (Dawid and Skene) [3] uses multiple noisy label sets to model a confusion matrix through an expectation–maximization [26] method, where the confusion matrix represents the crowd worker's probability of providing each label for instances belonging to each class. GLAD (Generative model of Labels, Abilities, and Difficulties) [32] uses mul-

multiple noisy label sets to build a probability model to infer the unknown true labels of instances, the labeling difficulty of instances, and the professional level of crowd workers. ZC (ZenCrowd) [4] and KOS [11] use multiple noisy label sets to model each crowd worker's reliability through an expectation–maximization method and a similar belief propagation method, respectively. PLAT (Positive Label frequency Threshold) [42] uses multiple noisy label sets to model the decision thresholds. GTIC [41] uses multiple noisy label sets to cluster all instances into different clusters, and then maps each cluster to a specific class.

The second category uses data features to help to infer unknown true labels from multiple noisy label sets. For example, [20] proposes a fine grained truth discovery model that uses data features and multiple noisy label sets to build a probabilistic model to simultaneously estimate topic expertise and infer true labels. [23] uses the potential true label as a single hidden layer, then uses the multiple noisy label set as the output layer, and finally uses data features to train the deep neural network (DNN) through backpropagation. BLCC (BiLayer Collaborative Clustering) [40] first generates concept-level features from multiple noisy label sets and conducts clustering on concept-level features to infer the initially integrated labels. Then, BLCC conducts another clustering on the physical-level features (data features) and uses the integrated labels obtained at the conceptual layer to infer the class membership of each cluster in the physical layer. Finally, the uncertainties of instances at the physical layer and the conceptual layer are calculated separately. For an instance with a small uncertainty at the physical layer and a large uncertainty at the conceptual layer, the label obtained at the physical layer is assigned as its final integrated label. [27] uses data features and multiple noisy label sets to evaluate each crowd worker's labeling quality on different instances and thus proposes four weighted soft MV methods. [36] introduces an active crowdsourcing learning strategy to compute the crowd consensus, which also uses data features and multiple noisy label sets. MNLDP (Multiple Noisy Label Distribution Propagation) [7] first uses multiple noisy label sets to calculate each instance's multiple noisy label distribution. Then, MNLDP uses data features to construct a feature space according to the overlapped local linear neighborhood patches and calculates each patch's edge weights through standard quadratic programming. Finally, each instance obtains a part of multiple noisy label distributions from its nearest neighbors while maintaining a part of its own. According to the final label distributions, the integrated labels of instances are obtained. [15] uses data features and multiple noisy label sets to combine the advantages of the DNN and the probabilistic graphics model and proposes a fully bayesian deep generation crowdsourcing model (BayesDGC). BayesDGC learns each crowd worker's confusion matrix and the distribution of potential true labels through end-to-end training.

3. Label Augmented and Weighted Majority Voting

3.1. Motivation

As mentioned above, many label integration methods have been proposed from different aspects. Although they generally perform better than MV, for better application in the real world, they still need to be further improved. In this paper, we argue that when the label quality is higher than random classification, the more the number of labels, the better the performance of label integration methods.

Now, let us illustrate our argument. For convenience, we first assume that each label has the same quality p and labeling type is binary. Here, the quality means the probability that the label is the true label. When the number of labels of each instance is $2n + 1$, the quality p' of the integrated label obtained by MV, can be calculated by:

$$p' = \sum_{i=0}^n \binom{2n+1}{i} p^{2n+1-i} (1-p)^i, \quad (1)$$

which is the sum of the probabilities of having fewer wrong labels than correct ones (i represents the number of wrong labels).

According to Eq. (1), Fig. 1 shows the relationship between the quality of the integrated label, the label quality and the number of labels. From this figure, we can see that when the label quality is greater than 0.5 and the number of labels is enough, the quality of the integrated label can get nearly close to 1. This indicates that when the label quality is higher than random classification and the number of labels is enough, even the simplest MV, the performance can be perfect. However, in real-world crowdsourcing scenarios, each instance cannot obtain enough labels for saving costs. Therefore, we consider augmenting each instance's multiple noisy label set by the labels of instances belonging to the same class as each instance. How to find enough instances with the same class for each instance to augment its multiple noisy label set is the focus of this paper. Inspired by KNN, we use the K -nearest neighbors algorithm to find each instance's K -nearest neighbors (including itself) that are most likely to belong to the same class as each instance and merge their multiple noisy label sets to obtain its augmented multiple noisy label set. Then, the labels from different neighbors are weighted by the distances and the label similarities between each instance and its neighbors to increase the impact of neighbors that are more likely to belong to the same class as each instance. Finally, the integrated labels are inferred by weighted MV. We call our proposed label integration method as label augmented and weighted majority voting (LAWMV).

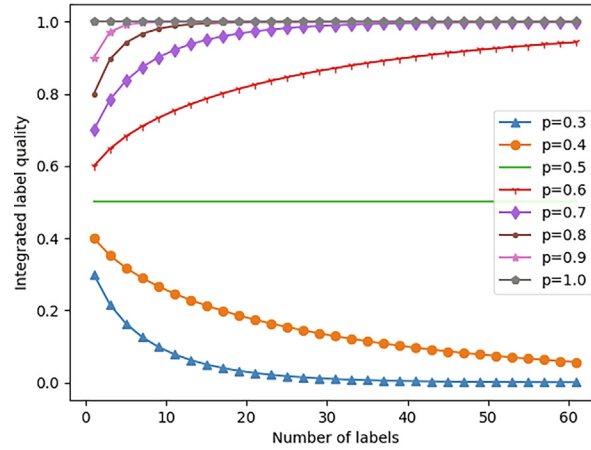


Fig. 1. The relationship between the quality of the integrated label, the label quality and the number of labels.

3.2. Overall framework of LAWMV

Before introducing the framework of LAWMV, we first define the relevant symbols. A crowdsourced dataset is commonly represented by a set $D = \{(\mathbf{x}_i, \mathbf{L}_i)\}_{i=1}^N$, where each instance $\mathbf{x}_i$ corresponds to a multiple noisy label set $\mathbf{L}_i = \{l_{ir}\}_{r=1}^R$. l_{ir} represents the label of $\mathbf{x}_i$ labeled by the crowd worker u_r ($r = 1, 2, \dots, R$) and takes value from the set $\{-1, c_1, c_2, \dots, c_Q\}$. Q represents the number of classes and -1 represents that $\mathbf{x}_i$ is not labeled by the corresponding crowd worker u_r . The purpose of label integration is to infer the integrated label $\hat{y}_i$ for each instance from its multiple noisy label set.

The overall framework of LAWMV is shown in Fig. 2. From it, we can see that the learning process of LAWMV is that, for each instance $\mathbf{x}_i$, we first perform label augmentation, then perform label weighting, and at last obtain its integrated label $\hat{y}_i$ by weighted MV. In the label augmentation part, we first find the K -nearest neighbors (including itself) for each instance $\mathbf{x}_i$ using the KNN algorithm. Then, we merge the multiple noisy label sets of each instance's K -nearest neighbors to obtain its augmented multiple noisy label set $\{\mathbf{L}_{i1}, \mathbf{L}_{i2}, \dots, \mathbf{L}_{iK}\}$. In the label weighting part, we first use the distances between each instance $\mathbf{x}_i$ and its neighbors to calculate weightings $\{w_{ik}^1\}_{k=1}^K$. Then, we use the label similarities between each instance $\mathbf{x}_i$ and its neighbors to calculate weightings $\{w_{ik}^2\}_{k=1}^K$. Finally, we combine $\{w_{ik}^1\}_{k=1}^K$ and $\{w_{ik}^2\}_{k=1}^K$ to assign weightings $\{w_{ik}\}_{k=1}^K$ for labels from different neighbors.

Now, two key issues remain to be resolved: 1) How to use the KNN algorithm to find each instance's K -nearest neighbors for augmenting each instance's multiple noisy label set. 2) How to use the distances and the label similarities between each instance and its neighbors to weight labels from different neighbors. We discuss these two issues separately in the following two subsections.

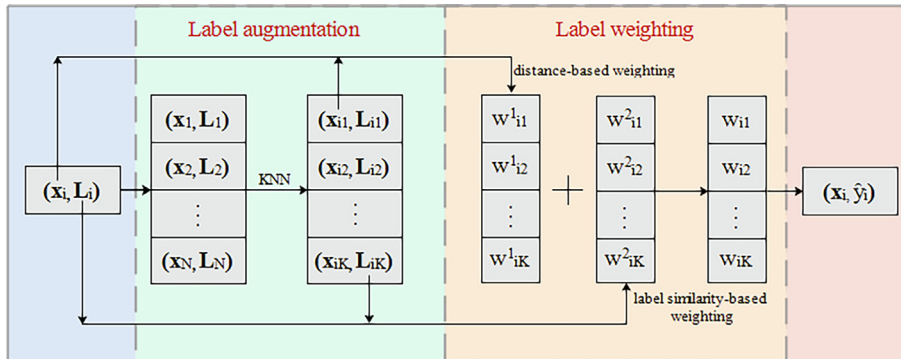


Fig. 2. The overall framework of LAWMV. It is mainly composed of label augmentation and label weighting. In the label augmentation part, we find the K -nearest neighbors (including itself) of $\mathbf{x}_i$ using the KNN algorithm and then merge their multiple noisy label sets to obtain its augmented multiple noisy label set. In the label weighting part, we use the distances and the label similarities between $\mathbf{x}_i$ and its neighbors to assign weightings $\{w_{ik}\}_{k=1}^K$ for labels from different neighbors.

3.3. Label augmentation

In this section, we will discuss in detail how to use the KNN algorithm to find each instance's K -nearest neighbors for augmenting its multiple noisy label set. We first briefly introduce the traditional KNN. For a test instance, the distance between it and each instance in a training set is first calculated. Then, the K -nearest neighbors that the distance to the test instance is closest are found. Finally, the test instance's class label is predicted according to class labels of its K -nearest neighbors. In order to apply it to crowdsourcing scenarios, the KNN used in our method is slightly different from the traditional KNN. In our method, for each instance in a dataset, its K -nearest neighbors (including itself) are found in the dataset. After this, multiple noisy label sets of each instance's K -nearest neighbors are merged to obtain its augmented multiple noisy label set. The calculation of the distance is crucial for finding each instance's K -nearest neighbors.

At present, many distance functions have been proposed. Here, we use the commonly used Heterogeneous Euclidean-Overlap Metric (HEOM) [33]. HEOM uses normalized euclidean distance for numerical features and overlap metric for nominal features. This function defines the distance between the a -th feature values of two instances $\mathbf{x}_i$ and $\mathbf{x}_j$ as:

$$d_a(\mathbf{x}_i, \mathbf{x}_j) = \begin{cases} 1, & \text{if } x_{ia} \text{ or } x_{ja} \text{ is unknown, else} \\ \text{overlap}(x_{ia}, x_{ja}), & \text{if } A_a \text{ is nominal, else} \\ \text{rn_diff}(x_{ia}, x_{ja}) \end{cases}, \quad (2)$$

where A_a represents the a -th feature.

If any feature value is unknown, a maximum feature distance of 1 is returned. The *overlap* and the range-normalized difference *rn_diff* are defined as:

$$\text{overlap}(x_{ia}, x_{ja}) = \begin{cases} 0, & \text{if } x_{ia} = x_{ja} \\ 1, & \text{otherwise} \end{cases}, \quad (3)$$

$$\text{rn_diff}(x_{ia}, x_{ja}) = \frac{|x_{ia} - x_{ja}|}{\max_a - \min_a}, \quad (4)$$

where $\max_a - \min_a$ is used to normalize numerical feature values, $\max_a$ and $\min_a$ represent the maximum and minimum values of the a -th feature observed in the dataset, respectively. The overall distance $d(\mathbf{x}_i, \mathbf{x}_j)$ between two instances $\mathbf{x}_i$ and $\mathbf{x}_j$ is defined as:

$$d(\mathbf{x}_i, \mathbf{x}_j) = \sqrt{\sum_a^m d_a(\mathbf{x}_i, \mathbf{x}_j)^2}, \quad (5)$$

where m is the number of features (not including the class).

For each instance $\mathbf{x}_i$, we first calculate the distances between it and all instances (including itself) in the dataset by Eqs. (2)–(5). Then these instances are sorted according to the distances from small to large to find the K -nearest neighbors $\{\mathbf{x}_{ik}\}_{k=1}^K$ of $\mathbf{x}_i$. Finally, all neighbors' multiple noisy label sets are merged to obtain $\mathbf{x}_i$'s augmented multiple noisy label set $\{\mathbf{L}_{i1}, \mathbf{L}_{i2}, \dots, \mathbf{L}_{iK}\}$. However, due to the limitations of KNN, the K -nearest neighbors cannot be guaranteed to belong to the same class as $\mathbf{x}_i$. Therefore, in Section 3.4, we use the distances and the label similarities between $\mathbf{x}_i$ and its neighbors to weight labels from different neighbors to further improve the effectiveness of the augmented multiple noisy label set.

3.4. Label weighting

In this section, we will discuss how to use the distances and the label similarities between each instance and its neighbors to weight labels from different neighbors.

The distance carried by each neighbor is obtained in the label augmentation part. This distance is used to measure the similarity between each instance and its neighbor. The larger the distance, the less similar between an instance and its neighbor, and the less likely the neighbor belongs to the same class as the instance. Therefore, we hope to give less weight to neighbors' labels with large distances to reduce the influence of neighbors' labels of different classes from the instance. For this reason, we define the weight w_{ik}^1 of labels of the k -th neighbor of $\mathbf{x}_i$ as:

$$w_{ik}^1 = 1 - \frac{d(\mathbf{x}_i, \mathbf{x}_{ik})}{d(\mathbf{x}_i, \mathbf{x}_{iK})}, \quad (6)$$

where $d(\mathbf{x}_i, \mathbf{x}_{ik})$ represents the distance between $\mathbf{x}_i$ and its k -th neighbor $\mathbf{x}_{ik}$, and $d(\mathbf{x}_i, \mathbf{x}_{iK})$ represents the distance between instance $\mathbf{x}_i$ and its K -th neighbor $\mathbf{x}_{iK}$ with the largest distance. This means that labels from the K -th neighbor are given a weight w_{iK}^1 of 0.

In addition, we further weight the labels of K -nearest neighbors by the label similarities between each instance and its K -nearest neighbors. Here, the label similarity means that the proportion of crowd workers who give the same label to an instance and its neighbor among the crowd workers who label them. We think that crowd workers tend to give the same labels to instances belonging to the same class. Therefore, the label similarity between each instance and its neighbor is lar-

ger, the neighbor is more similar to the instance. That is, the neighbor is more likely to belong to the same class as the instance. In order to better explain this phenomenon, we give a simple example here. For $\mathbf{x}_1$, there are two neighbors $\mathbf{x}_2$ and $\mathbf{x}_3$. $\mathbf{x}_1$ is labeled as classes c_1, c_2 , and c_3 by workers u_1, u_2 , and u_3 , respectively. $\mathbf{x}_2$ is labeled as classes c_2, c_3 , and c_4 by workers u_2, u_3 , and u_4 , respectively. $\mathbf{x}_3$ is labeled as classes c_4, c_2 , and c_3 by workers u_1, u_2 , and u_3 , respectively. According to the above description, the label similarity between $\mathbf{x}_2$ and $\mathbf{x}_1$ is $2/2 = 1$, and the label similarity between $\mathbf{x}_3$ and $\mathbf{x}_1$ is $2/3 = 0.67$. $\mathbf{x}_2$ is more similar to the instance $\mathbf{x}_1$ than $\mathbf{x}_3$, so labels from $\mathbf{x}_2$ should be given more weight. For this reason, we define the weight w_{ik}^2 of labels of the k -th neighbor of $\mathbf{x}_i$ as:

$$w_{ik}^2 = \begin{cases} 0, & \text{if } \sum_{r=1}^R (l_{ir} \neq -1 \wedge l_{ikr} \neq -1) = 0 \\ \frac{\sum_{r=1}^R \delta(l_{ir}, l_{ikr})}{\sum_{r=1}^R (l_{ir} \neq -1 \wedge l_{ikr} \neq -1)}, & \text{otherwise} \end{cases}, \quad (7)$$

where l_{ikr} represents the label of the k -th neighbor of $\mathbf{x}_i$ labeled by the crowd worker u_r .

After calculating w_{ik}^1 and w_{ik}^2 , we make a simple linear combination of them, and thus the final weight w_{ik} of each neighbor's labels is defined as:

$$w_{ik} = \frac{w_{ik}^1 + w_{ik}^2}{2}. \quad (8)$$

Finally, we apply the obtained weights to the augmented multiple noisy label set of each instance to calculate each label's posterior probability. Then, the label with the highest posterior probability of each instance is taken as its integration label $\hat{y}_i$, which can be expressed as:

$$\hat{y}_i = \arg \max_{c_q \in \{c_1, c_2, \dots, c_Q\}} \sum_{k=1}^K \sum_{r=1}^R \delta(l_{ikr}, c_q) \cdot w_{ik}. \quad (9)$$

3.5. K value setting

As we all know, the parameter K of the traditional distance-weighted KNN generally takes a small fixed value, and its performance is not sensitive to the value of K . We think this is mainly due to two points: 1) Each neighbor is weighted by distance. 2) Each neighbor's class label (called the true label in crowdsourcing) is used to predict the test instance's. However, our method is different from this. Although each neighbor is also weighted, each instance uses the neighbor's multiple noisy labels, which are noisy and different from the true label. Therefore, for the value of K in our method, a larger value should be taken as much as possible to reduce the influence of the noise labels in the multiple noisy label set. However, there is a problem. Assuming that the instance $\mathbf{x}_1$ belongs to c_1 , there are only 20 instances belonging to c_1 in the dataset. Obviously, setting K to a value greater than 20 is inappropriate. In the crowdsourcing scenarios, we don't know which class the instance belongs to, let alone the number of instances of the class it belongs to. We only know the number of instances N and the number of classes Q in the dataset. Therefore, we estimate the value of K by the following equation:

$$K = b \cdot \frac{N}{Q}, \quad (10)$$

where $\frac{N}{Q}$ roughly estimates the number of instances of each class. b is a hyperparameter, which is used to scale $\frac{N}{Q}$. Based on our experimental experience, it is better to set b to less than 1. In our experiments, we set b to 0.5.

3.6. Time complexity analysis

Now, the learning process of LAWMV is described in detail in Algorithm 1. First, line 1 calculates the value of K . Its time complexity is $O(1)$. And then, the integrated label of each instance $\mathbf{x}_i$ is inferred in lines 2–14. Its time complexity is mainly divided into three parts: 1) Label augmentation in lines 3–7. 2) Label weighting in lines 8–12. 3) Label inference in line 13. In the first part, to augment the multiple noisy label set of the instance $\mathbf{x}_i$, we first need to calculate the distances from all instances in the dataset to the instance $\mathbf{x}_i$. The time complexity of this process is $O(mN)$, where m is the number of features and N is the number of instances. Then we need to find the K -nearest neighbors with the smallest distance, which is achieved by the accepted best performance quick sorting algorithm with average time complexity of $O(N \log N)$. The first part's time complexity is $O(mN + N \log N)$. In the second part, we need to calculate the weight of labels of each neighbors. The time complexity is $O(KR)$, where R is the number of crowd workers. In the third part, we need to infer its integrated label for instance $\mathbf{x}_i$. The time complexity is $O(KR)$. Therefore, for each instance $\mathbf{x}_i$, the time complexity is $O((N \log N + mN) + KR + KR)$, and

thus the whole time complexity of LAWMV is $O(1 + N((N \log N + mN) + KR + KR))$. If only the highest order terms are taken, the time complexity of LAWMV is $O(N^2(\log N + m))$.

Algorithm 1: LAWMV

Input: $D = \{(\mathbf{x}_i, \mathbf{L}_i)\}_{i=1}^N$ -a crowdsourced dataset; b -a predefined parameter
Output: $\{\hat{y}_i\}_{i=1}^N$ -the integrated labels
1: Calculate the number of nearest neighbors K using Eq. (10)
2: **for** $i = 1$ to N **do**
3: **for** $j = 1$ to N **do**
4: Calculate the distance $d(\mathbf{x}_i, \mathbf{x}_j)$ between $\mathbf{x}_i$ and $\mathbf{x}_j$ by Eqs. (2)–(5)
5: **end for**
6: Find $\mathbf{x}_i$'s K -nearest neighbors $\{\mathbf{x}_{ik}\}_{k=1}^K$ by sorting the distances
7: Merge K -nearest neighbors' multiple noisy label sets to obtain $\mathbf{x}_i$'s augmented multiple noisy label set $\{\mathbf{L}_{i1}, \mathbf{L}_{i2}, \dots, \mathbf{L}_{iK}\}$.
8: **for** $k = 1$ to K **do**
9: Use the distance to calculate the weight w_{ik}^1 of the k -th neighbor's labels by Eq. (6)
10: Use the label similarity to calculate the weight w_{ik}^2 of the k -th neighbor's labels by Eq. (7)
11: Use w_{ik}^1 and w_{ik}^2 to calculate the final weight w_{ik} of the k -th neighbor's labels by Eq. (8)
12: **end for**
13: Infer the integrated label $\hat{y}_i$ of $\mathbf{x}_i$ by Eq. (9)
14: **end for**
15: Return $\{\hat{y}_i\}_{i=1}^N$

4. Experiments and Results

The purpose of this section is to evaluate the performance of our proposed LAWMV. Therefore, we compare LAWMV with MV, ZC, KOS, GTIC, IWMV, and MNLDP in terms of the integration accuracy and classification accuracy. Here, the integration accuracy represents the proportion of instances in the dataset where the integrated label is equal to the true label. The classification accuracy represents the proportion of instances in the dataset where the predicted label is equal to the true label. In our experiments, we implemented LAWMV, IWMV, and MNLDP on the CEKA [39] platform. And in LAWMV, the parameter b is set to 0.5. In addition, for MV, ZC, KOS and GTIC, we used the existing implementations and parameter settings on the CEKA platform.

4.1. Experiments on Simulated Datasets

In this subsection, we conduct our experiments on the whole 34 simulated datasets from the CEKA platform. Table 1 shows the details of these datasets that represent a wide range of domains and data characteristics. Since some datasets have missing values and some competitors cannot handle missing values. Therefore, in our experiment, we use the mean of numerical feature values or the modes of the nominal feature values from the available data to replace all missing feature values by the unsupervised feature filter ReplaceMissingValues in the WEKA [34] platform. After replacing all missing feature values, we simulate the crowdsourcing process to obtain the multiple noisy label sets of all instances. We first hide the true labels of all instances. Then, we simulate crowd workers to label the instances and the probability of labeling each instance as its original true label is p and the probability of other labels depends on their proportions in the dataset. In our experiments, we randomly generate each crowd worker's labeling quality p from a uniform distribution in the interval $[0.3, 0.9]$, and the number of crowd workers is set to 5.

After obtaining five labels of each instance, we first apply label integration methods, including MV, ZC, KOS, GTIC, IWMV, MNLDP and our proposed LAWMV, to infer the integrated labels. Then, we use the training set with integrated labels to build a classifier and use the test set with true labels to evaluate its performance by ten-fold cross-validation. Here, all label integration methods use C4.5 (J48) [21] as the target classifier. In order to reduce the fluctuation caused by the randomness of the simulation process, each experiment is repeated ten times independently, and the integration accuracies and classification accuracies are averaged, respectively.

Tables 2 and 4 show the detailed comparison results in terms of the integration accuracy and classification accuracy, respectively. The last row of the two tables is the average value of each method on all datasets, indicating the overall performance of each method, and the highest average value is shown in bold. Based on the experimental results in Tables 2 and 4, we compare each pair of methods by performing the Wilcoxon signed-rank test [5,8]. Tables 3 and 5 summarize the Wilcoxon test results. In Tables 3 and 5, the symbol • indicates that the method in the row significantly outperforms the method

Table 1

The description of 34 simulated datasets.

Dataset	Instances	Features	Classes	Missing values	Feature type
anneal	898	38	6	yes	hybrid
audiology	226	69	24	yes	nominal
autos	205	25	7	yes	hybrid
balance-scale	625	4	3	no	numeric
biodeg	1055	41	2	no	numeric
breast-cancer	286	9	2	yes	nominal
breast-w	699	9	2	yes	numeric
car	1728	6	4	no	nominal
credit-a	690	15	2	yes	hybrid
credit-g	1000	20	2	no	hybrid
diabetes	768	8	2	no	numeric
heart-c	303	13	5	yes	hybrid
heart-h	294	13	5	yes	hybrid
heart-statlog	270	13	2	no	numeric
hepatitis	155	19	2	yes	hybrid
horse-colic	368	22	2	yes	hybrid
hypothyroid	3772	29	4	yes	hybrid
ionosphere	351	34	2	no	numeric
iris	150	4	3	no	numeric
kr-vs-kp	3196	36	2	no	nominal
labor	57	16	2	yes	hybrid
letter	20000	16	26	no	numeric
lymph	148	18	4	no	hybrid
mushroom	8124	22	2	yes	nominal
segment	2310	19	7	no	numeric
sick	3772	29	2	yes	hybrid
sonar	208	60	2	no	numeric
spambase	4601	57	2	no	numeric
tic-tac-toe	958	9	2	no	nominal
vehicle	846	18	4	no	numeric
vote	435	16	2	yes	nominal
vowel	990	13	11	no	hybrid
waveform	5000	40	3	no	numeric
zoo	101	17	7	no	hybrid

in the corresponding column, and the symbol $\circ$ indicates the exact opposite of that indicated by the symbol $\bullet$. The significance levels of the lower and upper diagonals are $\alpha = 0.05$ and $\alpha = 0.1$, respectively. These experimental results in Tables 2–5 fully verify the effectiveness of our proposed LAWMV in improving the label quality and model quality. We can summarize the following highlights:

1. The average integration accuracies of LAWMV and MNLDP on all datasets are 82.89% and 80.12%, respectively, which are much higher than those of MV (74.03%), ZC (73.87%), KOS (76.82%), GTIC (70.49%) and IWMV (77.29%). Different from the other five methods, LAWMV and MNLDP explore neighbors' labels and thus achieve better performance in terms of the integration accuracy.
2. The average integration accuracy of our proposed LAWMV (82.89%) on all datasets is much higher than that of MNLDP (80.12%). Compared with MNLDP which sets the number of neighbors to a small and fixed value, our LAWMV uses neighbors' labels as much as possible and combines the label similarity in crowdsourcing to weight labels from different neighbors, thus our LAWMV has better performance in terms of the integration accuracy.
3. The average classification accuracy of our proposed LAWMV on all datasets is 77.47%, which is also much higher than those of MV (74.26%), ZC (72.85%), KOS (73.75%), GTIC (70.21%), IWMV (75.46%) and MNLDP (76.11%). In theory, the models trained on datasets with higher integration accuracy can achieve higher classification accuracy, thus our LAWMV performs best in classification accuracy. This also verifies the effectiveness of our proposed LAWMV again.
4. Based on the Wilcoxon test results, LAWMV significantly outperforms all of its competitors not only in terms of the label quality but also in terms of the model quality.

4.2. Experiments on Real-World Datasets

In this subsection, to further evaluate the performance of LAWMV, we conduct our experiments on the whole two real-world crowdsourced datasets: "Income" and "Leaves" from the CEKA platform. The "Income" dataset is a classic binary crowdsourced dataset that contains 600 instances, 6000 labels and 10 features. And it has 4 numeric features and 6 nominal features, and no missing feature values. This dataset was labeled by 67 crowd workers through the online platform Amazon Mechanical Turk (AMT), and each instance's labels were from 10 different crowd workers. The "Leaves" dataset is a classic

Table 2

The integration accuracy (%) comparisons.

Dataset	MV	ZC	KOS	GTIC	IWMV	MNLDP	LAWMV
anneal	81.65	76.12	88.64	53.76	86.22	88.76	83.75
audiology	71.73	66.15	59.38	67.26	74.47	73.67	73.54
autos	81.07	72.63	74.24	72.78	85.76	82.73	86.63
balance-scale	69.54	75.20	75.01	69.18	72.83	75.44	80.75
biodeg	71.36	73.92	78.07	71.36	74.23	74.35	78.54
breast-cancer	69.20	73.08	79.72	69.20	74.23	70.70	76.19
breast-w	63.82	66.80	71.77	63.82	66.05	72.75	81.13
car	74.42	73.55	83.15	70.87	80.12	80.07	79.35
credit-a	69.58	74.39	77.12	69.58	74.93	74.58	82.26
credit-g	71.22	77.72	79.92	71.22	76.74	70.26	75.94
diabetes	72.28	74.73	75.44	72.28	76.72	75.07	78.09
heart-c	75.02	77.23	84.22	44.46	75.02	79.01	83.60
heart-h	80.58	87.18	87.48	48.47	80.58	84.25	89.08
heart-statlog	66.07	67.44	70.48	66.07	68.00	68.52	71.56
hepatitis	64.77	67.35	67.10	64.77	70.71	71.68	76.26
horse-colic	67.45	70.16	74.57	67.45	70.49	68.42	75.60
hypothyroid	75.61	72.41	85.29	64.99	79.67	89.63	92.18
ionosphere	66.01	70.09	66.75	66.01	70.14	71.28	72.14
iris	81.93	79.87	83.00	81.47	85.87	95.87	98.40
kr-vs-kp	72.12	78.65	78.55	72.12	75.95	80.68	80.70
labor	68.95	67.72	61.75	67.19	67.54	71.93	74.91
letter	93.78	75.18	75.90	93.56	94.92	99.57	98.60
lymph	71.49	79.86	79.80	71.82	73.11	74.59	78.99
mushroom	66.36	68.57	79.18	66.36	68.62	79.85	82.41
segment	87.04	72.50	80.12	87.01	90.19	97.62	96.49
sick	69.84	73.00	77.30	69.84	72.08	80.17	87.25
sonar	72.36	76.92	76.15	72.36	77.02	77.64	77.21
spambase	73.59	79.98	85.20	73.59	77.50	80.11	84.28
tic-tac-toe	67.31	67.31	75.19	67.31	67.31	65.81	71.33
vehicle	80.84	79.15	78.33	80.96	83.89	87.26	86.87
vote	73.31	77.91	73.24	73.31	78.44	81.63	87.03
vowel	89.24	72.33	71.53	88.95	91.29	99.12	92.00
waveform	77.72	75.79	81.08	77.39	83.59	89.28	94.25
zoo	79.60	70.59	77.23	79.80	83.56	91.88	90.99
Average	74.03	73.87	76.82	70.49	77.29	80.12	82.89

Table 3

The integration accuracy (%) comparisons of the Wilcoxon test.

	MV	ZC	KOS	GTIC	IWMV	MNLDP	LAWMV
MV	-		◦	•	◦	◦	◦
ZC		-	◦	•	◦	◦	◦
KOS	•	•	-	•			◦
GTIC	◦	◦	◦	-	◦	◦	◦
IWMV	◦	•		•	-	◦	◦
MNLDP	•	•		•	•	-	◦
LAWMV	•	•	•	•	•	•	-

multiclass crowdsourced dataset that contains 6 different classes, 384 instances and 3840 labels, and it has only 64 numeric features and no missing feature values. This dataset was labeled by 83 crowd workers through the online platform AMT, and each instance's labels were from 10 different crowd workers.

Fig. 3 shows the performance of seven methods MV, ZC, KOS, GTIC, IWMV, MNLDP and LAWMV on the “Income” dataset, which is measured by the integration accuracy and classification accuracy. We can see that our proposed LAWMV is obviously better than other competitors in terms of the integration accuracy and classification accuracy. Specifically, the integration accuracy of our LAWMV (74.00%) is much higher than those of MV (71.17%), ZC (72.33%), KOS (71.33%), GTIC (71.83%), IWMV (72.00%) and MNLDP (71.00%) and the classification accuracy of our LAWMV (70.47%) is also much higher than those of MV (68.72%), ZC (69.38%), KOS (69.51%), GTIC (69.42%), IWMV (69.15%) and MNLDP (69.23%). This demonstrates the effectiveness of our proposed LAWMV.

Fig. 4 shows the performance of the above seven methods on the “Leaves” dataset. We can see that our proposed LAWMV is obviously better than other competitors in terms of the integration accuracy and classification accuracy. Specifically, the integration accuracy of our LAWMV (66.15%) is much higher than those of MV (64.32%), ZC (64.58%), KOS (63.80%), GTIC (62.24%), IWMV (64.58%) and MNLDP (65.10%) and the classification accuracy of our LAWMV (55.63%) is also much higher

Table 4

The classification accuracy (%) comparisons.

Dataset	MV	ZC	KOS	GTIC	IWMV	MNLDP	LAWMV
anneal	88.37	81.79	86.54	55.92	90.12	88.44	80.54
audiology	65.00	61.32	47.55	62.04	67.02	64.76	67.26
autos	66.55	59.06	53.35	59.19	68.82	70.59	70.88
balance-scale	68.67	71.75	73.38	67.58	70.17	71.90	77.92
biodeg	67.30	68.25	71.86	67.38	68.80	70.78	72.89
breast-cancer	63.36	66.11	70.29	63.18	66.09	67.94	70.09
breast-w	76.00	76.13	74.39	75.80	75.98	75.96	82.64
car	81.39	79.31	84.96	80.40	84.65	76.16	72.83
credit-a	70.26	71.32	75.04	70.22	72.30	75.46	79.61
credit-g	62.80	64.87	66.93	63.06	64.70	64.86	66.63
diabetes	66.73	67.57	67.02	66.77	67.67	69.44	69.96
heart-c	67.30	67.90	72.62	31.99	66.94	71.23	75.98
heart-h	76.06	77.79	77.76	33.63	75.94	78.61	79.83
heart-statlog	62.03	62.65	63.80	62.00	63.43	64.25	68.06
hepatitis	62.72	63.36	62.95	62.34	66.10	69.82	71.97
horse-colic	65.96	67.97	70.45	65.88	68.57	67.17	72.93
hypothyroid	94.69	87.40	97.89	85.27	96.36	93.63	92.29
ionosphere	71.83	74.57	68.89	71.96	74.39	70.91	71.20
iris	89.78	84.33	87.97	86.43	88.95	94.31	94.65
kr-vs-kp	85.49	88.78	88.47	85.53	86.62	84.31	86.89
labor	66.98	64.53	57.81	65.79	64.82	66.98	71.25
letter	87.27	77.04	75.28	87.20	87.36	87.94	87.52
lymph	64.89	70.79	71.52	65.29	64.89	66.78	71.94
mushroom	83.94	83.78	94.38	83.93	83.85	85.64	82.89
segment	94.34	83.25	88.87	94.17	94.99	96.18	94.42
sick	85.11	85.68	88.77	85.04	85.59	85.34	88.00
sonar	61.43	63.35	61.70	61.14	63.16	64.47	64.56
spambase	86.15	88.21	89.85	86.23	87.49	82.29	83.88
tic-tac-toe	66.97	66.75	72.58	67.01	66.74	62.67	68.14
vehicle	63.89	62.05	60.60	62.87	64.87	70.67	68.61
vote	85.06	86.19	76.97	85.02	86.19	84.42	86.56
vowel	76.29	64.42	61.93	75.79	76.87	79.41	76.56
waveform	61.90	60.15	64.23	62.18	65.82	73.27	74.96
zoo	88.49	78.50	81.02	89.07	89.49	91.10	89.84
Average	74.26	72.85	73.75	70.21	75.46	76.11	77.47

Table 5

The classification accuracy (%) comparisons of the Wilcoxon test.

	MV	ZC	KOS	GTIC	IWMV	MNLDP	LAWMV
MV	-			•	◦	◦	◦
ZC		-			◦	◦	◦
KOS			-				◦
GTIC	◦			-	◦	◦	◦
IWMV	•	•		•	-		◦
MNLDP	•	•		•		-	◦
LAWMV	•	•	•	•	•	•	-

than those of MV (51.74%), ZC (53.07%), KOS (51.67%), GTIC (50.16%), IWMV (52.24%) and MNLDP (52.27%). This demonstrates the effectiveness of our proposed LAWMV again.

4.3. Parameter Sensitivity Analysis of LAWMV

In this paper, we use $\frac{N}{Q}$ roughly to estimate the number of instances of each class and use b to scale $\frac{N}{Q}$ to obtain the value of the number of neighbors K . Therefore, the larger the b , the larger the K , and the smaller the b , the smaller the K . We believe that as many neighbors as possible should be found for each instance in LAWMV to obtain its augmented multiple noisy label set to achieve better performance. However, when the number of neighbors exceeds the number of instances of the class to which the instance belongs, the performance of LAWMV decreases as K increases. In addition, the running time of LAWMV will also be increased as K increases. Therefore, the parameter b should not be set too large nor should it be set too small.

Fig. 5 shows the integration accuracy of LAWMV when b varies from 0.1 to 1.0 on the “Leaves” dataset. We can see that when b takes values from 0.1 to 0.4, the integration accuracy of LAWMV gradually increases as the number of neighbors increases. When b takes a value from 0.6 to 1.0, the integration accuracy of LAWMV gradually decreases as the number of

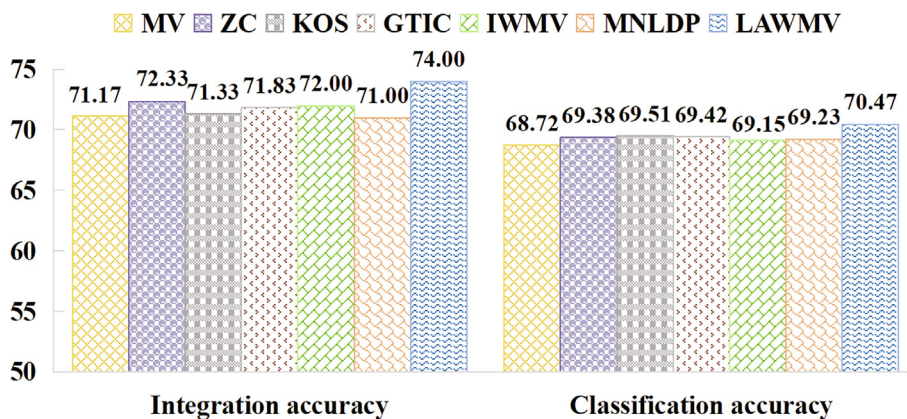


Fig. 3. The integration accuracy (%) and classification accuracy (%) comparisons for LAWMV versus MV, ZC, KOS, GTIC, IWMV and MNLDP on the "Income" dataset.

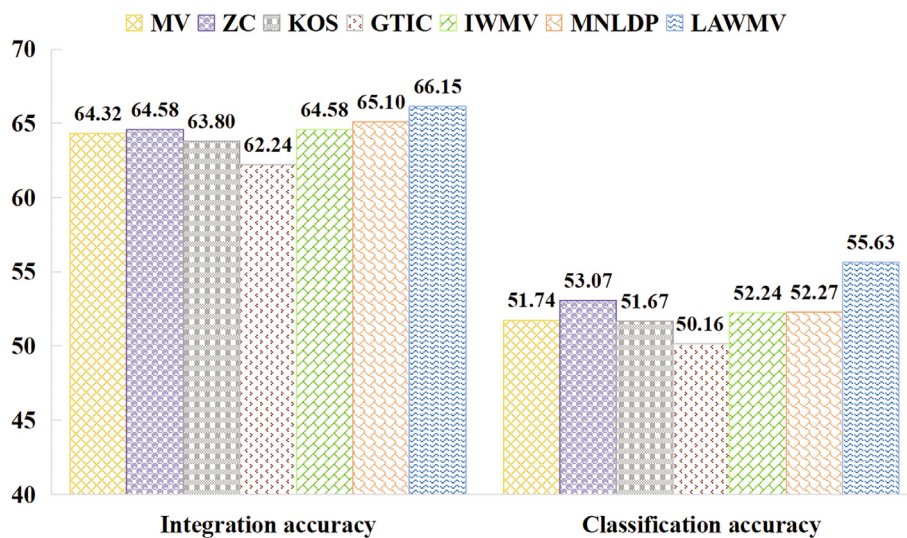


Fig. 4. The integration accuracy (%) and classification accuracy (%) comparisons for LAWMV versus MV, ZC, KOS, GTIC, IWMV and MNLDP on the "Leaves" dataset.

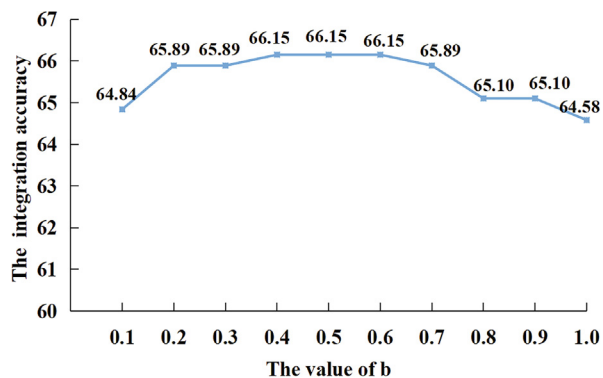


Fig. 5. The integration accuracy (%) comparisons for LAWMV on the "Leaves" dataset when b varies from 0.1 to 1.0.

neighbors increases. When b takes a value from 0.4 to 0.6, the performance of LAWMV reaches the best 66.15%. To guarantee the accuracy and efficiency of LAWMV, b was set to 0.5 in our experiments.

5. Conclusions and Future Work

This paper proposes a novel label integration method, simply LAWMV. LAWMV first uses the K -nearest neighbors (KNN) algorithm to find each instance's K -nearest neighbors (including itself) and merges their multiple noisy label sets to obtain its augmented multiple noisy label set. Then, the labels from different neighbors are weighted by the distances and the label similarities between each instance and its neighbors. Finally, the integrated label of each instance is inferred by weighted majority voting (MV). The extensive experiments on 34 simulated and two real-world crowdsourced datasets show that LAWMV significantly outperforms all of its state-of-the-art competitors in terms of the label quality and model quality. Our work provides a simple but effective label integration method that could be applied to a broad range of practical applications such as crowdsourced image recognition and text classification.

Seen from Eq. (9), for each instance, not only its own multiple noisy labels but also its augmented multiple noisy labels from the same neighbor all have the same weights. Namely, the multiple noisy labels from the same instance have the same quality, which is a little simple and rough. We believe that learning a different weight for each noisy label even from the same instance could further improve the current LAWMV's performance and make its advantages more stronger. This is the main direction of our future work.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The work was partially supported by Science and Technology Project of Hubei Province-Unveiling System (2021BEC007), Industry-University-Research Innovation Funds for Chinese Universities (2020ITA05008), and Foundation of Key Laboratory of Artificial Intelligence, Ministry of Education, P.R. China (AI2020002).

References

- [1] M. Buhrmester, T. Kwang, S.D. Gosling, Amazon's mechanical turk: A new source of inexpensive, yet high-quality, data?, *Perspect Psychol. Sci.* 6 (2011) 3–5.
- [2] P. Chen, H. Sun, Y. Fang, X. Liu, CONAN: A framework for detecting and handling collusion in crowdsourcing, *Inf. Sci.* 515 (2020) 44–63.
- [3] A.P. Dawid, A.M. Skene, Maximum likelihood estimation of observer error-rates using the em algorithm, *J. R. Stat.Soc. : Series C (Appl. Stat.)* 28 (1979) 20–28.
- [4] G. Demartini, D.E. Difallah, P. Cudré-Mauroux, Zencrowd: leveraging probabilistic reasoning and crowdsourcing techniques for large-scale entity linking, in: *Proceedings of the 21st international conference on World Wide Web*, 2012, pp. 469–478.
- [5] J. Demsar, Statistical comparisons of classifiers over multiple data sets, *J. Mach. Learn. Res.* 7 (2006) 1–30.
- [6] Y. Dong, L. Jiang, C. Li, Improving data and model quality in crowdsourcing using co-training-based noise correction, *Inf. Sci.* 583 (2022) 174–188.
- [7] L. Jiang, H. Zhang, F. Tao, C. Li, Learning from crowds with multiple noisy label distribution propagation, *IEEE Trans. Neural Networks Learn. Syst.* (2021), <https://doi.org/10.1109/TNNLS.2021.3082496>.
- [8] L. Jiang, L. Zhang, C. Li, J. Wu, A correlation-based feature weighting filter for naive bayes, *IEEE Trans. Knowl. Data Eng.* 31 (2019) 201–213.
- [9] L. Jiang, L. Zhang, L. Yu, D. Wang, Class-specific attribute weighted naive bayes, *Pattern Recognit.* 88 (2019) 321–330.
- [10] Kang, X., Yu, G., Domeniconi, C., Wang, J., Guo, W., Ren, Y., & Cui, L. (2021). Crowdsourcing with self-paced workers. In *IEEE International Conference on Data Mining, ICDM 2021*, Auckland, New Zealand, December 7–10, 2021 (pp. 280–289)..
- [11] D.R. Karger, S. Oh, D. Shah, Budget-optimal task allocation for reliable crowdsourcing systems, *Operations Res.* 62 (2014) 1–24.
- [12] C. Li, L. Jiang, W. Xu, Noise correction to improve data and model quality for crowdsourcing, *Eng. Appl. Artif. Intell.* 82 (2019) 184–191.
- [13] C. Li, V.S. Sheng, L. Jiang, H. Li, Noise filtering to improve data and model quality for crowdsourcing, *Knowl. Based Syst.* 107 (2016) 96–103.
- [14] Li, H., & Yu, B. (2014). Error rate bounds and iterative weighted majority voting for crowdsourcing. *arXiv preprint arXiv:1411.4086*.
- [15] S.-Y. Li, S.-J. Huang, S. Chen, Crowdsourcing aggregation with deep bayesian learning, *Sci. China-Inform. Sci.* 64 (2021).
- [16] Z. Li, J. Tang, Weakly supervised deep metric learning for community-contributed image retrieval, *IEEE Trans. Multimed.* 17 (2015) 1989–1999.
- [17] Z. Li, J. Tang, Weakly supervised deep matrix factorization for social image understanding, *IEEE Trans. Image Process.* 26 (2017) 276–288.
- [18] Z. Li, J. Tang, T. Mei, Deep collaborative embedding for social image understanding, *IEEE Trans. Pattern Anal. Mach. Intell.* 41 (2019) 2070–2083.
- [19] Z. Li, J. Tang, L. Zhang, J. Yang, Weakly-supervised semantic guided hashing for social image retrieval, *Int. J. Comput. Vis.* 128 (2020) 2265–2278.
- [20] Ma, F., Li, Y., Li, Q., Qiu, M., Gao, J., Zhi, S., Su, L., Zhao, B., Ji, H., & Han, J. (2015). Faticrowd: Fine grained truth discovery for crowdsourced data aggregation. In *Proceedings of the 21th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, Sydney, NSW, Australia, August 10–13, 2015 (pp. 745–754)..
- [21] J.R. Quinlan, C4.5: Programs for Machine Learning, Morgan Kaufmann, 1993.
- [22] Ramakrishna, A., Gupta, R., Grossman, R.B., & Narayanan, S.S. (2016). An expectation maximization approach to joint modeling of multidimensional ratings derived from multiple annotators. In *Interspeech* (pp. 1555–1559)..
- [23] F. Rodrigues, F.C. Pereira, Deep learning from crowds, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018, pp. 1611–1618.
- [24] Sheng, V.S., Provost, F.J., & Ipeirotis, P.G. (2008). Get another label? improving data quality and data mining using multiple, noisy labels. In *Proceedings of the 14th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, Las Vegas, Nevada, USA, August 24–27, 2008 (pp. 614–622)..
- [25] V.S. Sheng, J. Zhang, B. Gu, X. Wu, Majority voting and pairing with multiple noisy labeling, *IEEE Trans. Knowl. Data Eng.* 31 (2019) 1355–1368.
- [26] A.K. Singh, The EM algorithm and related statistical models, *Technometrics* 48 (2006) 148.
- [27] F. Tao, L. Jiang, C. Li, Label similarity-based weighted soft majority voting and pairing for crowdsourcing, *Knowl. Inf. Syst.* 62 (2020) 2521–2538.

- [28] F. Tao, L. Jiang, C. Li, Differential evolution-based weighted soft majority voting for crowdsourcing, *Eng. Appl. Artif. Intell.* 106 (2021) 104474.
- [29] Tu, J., Yu, G., Domeniconi, C., Wang, J., Xiao, G., & Guo, M. (2018). Multi-label answer aggregation based on joint matrix factorization. In *IEEE International Conference on Data Mining, ICDM 2018, Singapore, November 17–20, 2018* (pp. 517–526)..
- [30] J. Tu, G. Yu, J. Wang, C. Domeniconi, M. Guo, X. Zhang, Crowdwit: Crowdsourcing via joint modeling of workers and tasks, *ACM Trans. Knowl. Discov. Data* 15 (2021), 12:1–12:24.
- [31] S. Wang, D. Dang, Incentive mechanism for the listing item task in crowdsourcing, *Inf. Sci.* 512 (2020) 80–95.
- [32] J. Whitehill, T.-F. Wu, J. Bergsma, J. Movellan, P. Ruvolo, Whose vote should count more: Optimal integration of labels from labelers of unknown expertise, *Adv. Neural Inform. Processing Syst.* 22 (2009) 2035–2043.
- [33] D.R. Wilson, T.R. Martinez, Improved heterogeneous distance functions, *J. Artif. Intell. Res.* 6 (1997) 1–34.
- [34] I.H. Witten, E. Frank, M.A. Hall, *Data mining: practical machine learning tools and techniques*, 3rd Edition., Morgan Kaufmann, Elsevier, 2011.
- [35] W. Xu, L. Jiang, C. Li, Improving data and model quality in crowdsourcing using cross-entropy-based noise correction, *Inf. Sci.* 546 (2021) 803–814.
- [36] G. Yu, J. Tu, J. Wang, C. Domeniconi, X. Zhang, Active multilabel crowd consensus, *IEEE Trans. Neural Networks Learn. Syst.* 32 (2021) 1448–1459.
- [37] H. Zhang, L. Jiang, L. Yu, Class-specific attribute value weighting for naive bayes, *Inf. Sci.* 508 (2020) 260–274.
- [38] J. Zhang, Knowledge learning with crowdsourcing: A brief review and systematic perspective, *IEEE/CAA Journal of Automatica Sinica* 9 (2022) 749–762.
- [39] J. Zhang, V.S. Sheng, B. Nicholson, X. Wu, CEKA: a tool for mining the wisdom of crowds, *J. Mach. Learn. Res.* 16 (2015) 2853–2858.
- [40] J. Zhang, V.S. Sheng, J. Wu, Crowdsourced label aggregation using bilayer collaborative clustering, *IEEE Trans. Neural Networks Learn. Syst.* 30 (2019) 3172–3185.
- [41] J. Zhang, V.S. Sheng, J. Wu, X. Wu, Multi-class ground truth inference in crowdsourcing with clustering, *IEEE Trans. Knowl. Data Eng.* 28 (2016) 1080–1085.
- [42] J. Zhang, X. Wu, V.S. Sheng, Imbalanced multiple noisy labeling, *IEEE Trans. Knowl. Data Eng.* 27 (2015) 489–503.